Chapter 5 — Exam Notes

The Relational Data Model & Relational Database Constraints

Fundamentals of Database Systems — Elmasri & Navathe, 7th Ed.

⚠ Low exam weightage — focus on definitions, key types, constraint rules, and update violations

---

1. Relational Model — Core Concepts

1.1 What is a Relation?

- Proposed by **Dr. E.F. Codd** (IBM) in 1970 — earned him the ACM Turing Award.
- A relation is a mathematical concept based on set theory.
- Informally, a relation looks like a table of values with rows and columns.

1.2 Informal vs. Formal Terminology

Informal Term	Formal Term	Meaning
Table	Relation	The structure that holds data
Column Header	Attribute	Name/description of a data field
All possible column values	Domain	Set of all valid values for an attribute
Row	Tuple	A single data record
Table definition	Schema of a Relation	Structure/blueprint of the relation
Populated Table	State of the Relation	Actual data at a point in time

> ✓ *Exam Tip: This comparison table is a classic exam question. Memorise all six pairs.*

1.3 Schema (R)

- Notation: R(A1, A2, ..., An)
- R = name of the relation; A1...An = its attributes.
- Example: CUSTOMER(Cust-id, Cust-name, Address, Phone#)
- Each attribute has a domain — the set of valid values it can take.

> □ *Domain example: dom(Cust-id) = 6-digit numbers; dom(Cust-name) = varchar(25).*

1.4 Tuple

- An ordered set of values enclosed in angle brackets: <v1, v2, ..., vn>.
- Each value vi must be drawn from dom(Ai).
- Example (4-tuple for CUSTOMER): <632895, "John Smith", "101 Main St.", "(404) 894-2000">

1.5 Domain

- Has a logical definition (e.g., "USA_phone_numbers").
- Also has a data type/format (e.g., (ddd)ddd-dddd).
- Same domain can be used by multiple attributes with different roles.
  - Example: Domain 'Date' used as both 'Invoice-date' and 'Payment-date'.

1.6 Relation State r(R)

- A specific population (set of tuples) of the relation at a given time.
- Formally: r(R) ⊆ dom(A1) × dom(A2) × ... × dom(An) (Cartesian product)
- Example: If dom(A1) = {0,1} and dom(A2) = {a,b,c}, then r(R) could be { <0,a>, <0,b>, <1,c> }.

> □ *Relation state is also called a snapshot or instance (though 'instance' can also mean a single tuple).*

---

2. Characteristics of Relations

2.1 Tuple Ordering

- Tuples are NOT ordered — a relation is a set, not a list.
- Two tables with the same tuples in different orders represent the same relation state.

2.2 Attribute Ordering

- In the formal model, attributes are ordered: R(A1, A2, ..., An).
- An alternative 'self-describing' representation includes attribute names with values, e.g., t = { <name, "John">, <SSN, 123456789> } — no ordering needed here.

2.3 Atomic Values

- All values in a tuple must be atomic (indivisible).
- Each value must belong to the domain of its attribute.
- NULL is a special value representing unknown, not available, or inapplicable data.

> ✓*Exam Tip: The relational model does NOT allow a list or set as an attribute value — this is an inherent constraint.*

2.4 Tuple Notation

- t[Ai] or t.Ai — the value of attribute Ai in tuple t.
- t[Au, Av, ..., Aw] — the sub-tuple of t containing values of specified attributes.

---

3. Relational Integrity Constraints

Constraints are conditions that must hold on ALL valid relation states. There are three types of constraints:

Type	Scope	Description
Inherent / Implicit	Data Model	Based on the model itself (e.g., no list values)
Schema-based / Explicit	Schema (DDL)	Expressed via CREATE TABLE — keys, foreign keys, NOT NULL, domains
Application-based / Semantic	Application code	Enforced by programs; beyond the model's expressive power

3.1 Domain Constraint

- Every value in a tuple must come from the domain of its attribute.
- Or it can be NULL (if NULL is permitted for that attribute).

3.2 Key Constraints

Superkey

- A set of attributes SK such that no two distinct tuples have the same SK value.
- t1[SK] ≠ t2[SK] for all distinct t1, t2 in r(R).

Key (Candidate Key)

- A minimal superkey — removing any attribute breaks the uniqueness property.
- Every key is a superkey, but not every superkey is a key.

> □ *Example: CAR(State, Reg#, SerialNo, Make, Model, Year) Key1 = {State, Reg#} | Key2 = {SerialNo} {SerialNo, Make} is a superkey but NOT a key (not minimal).*

Primary Key

- When multiple candidate keys exist, one is chosen as the primary key.
- Primary key attributes are underlined in schema diagrams.
- General rule: choose the smallest candidate key.
- Uniquely identifies each tuple; also used by other relations to reference tuples.

Surrogate / Artificial Key

- A system-assigned sequential number used as a key when no natural key exists.

> ✓*Exam Tip: Know the difference: superkey ⊇ key ⊇ primary key. A primary key is one chosen candidate key.*

3.3 Entity Integrity Constraint

- The primary key (PK) of every relation cannot contain NULL in any tuple.
- Formal: t[PK] ≠ null for every tuple t in r(R).
- Applies to ALL attributes of a composite PK — none can be null.
- Other non-PK attributes may also be declared NOT NULL, but that is a separate constraint.

> ✓*Exam Tip: Primary key = never null. This is entity integrity.*

3.4 Referential Integrity (Foreign Key) Constraint

- Involves TWO relations: a referencing relation R1 and a referenced relation R2.
- R1 has a foreign key (FK) that references the primary key (PK) of R2.
- t1 in R1 references t2 in R2 when t1[FK] = t2[PK].

The Foreign Key Rule

- FK value in R1 must either:
  - (1) match an existing PK value in R2, OR
  - (2) be NULL (only if FK is not part of R1's own PK).
- Displayed in schema diagrams as a directed arc from R1.FK → R2 (or R2.PK).

> *Example (COMPANY schema): EMPLOYEE.Dno → DEPARTMENT.Dnumber An employee must belong to an existing department, or Dno is null.*
>
> *✓Exam Tip: The foreign key does NOT have to have the same name as the PK it references — only the domain must match.*

3.5 Semantic (Application-based) Constraints

- Cannot be expressed by the relational model alone.
- Example: "No employee may work more than 56 hours per week across all projects."
- Expressed in SQL-99 via CREATE TRIGGER or CREATE ASSERTION.

---

4. Relational Database Schema & State

4.1 Database Schema

- S = { R1, R2, ..., Rn } — a set of relation schemas + a set of integrity constraints (IC).
- COMPANY database example has 6 relations: EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT.

4.2 Database State (Snapshot)

- DB = { r1, r2, ..., rm } — a set of relation states satisfying all integrity constraints in IC.
- Also called a relational database snapshot.
- A state that violates any constraint is an invalid state.
- The state changes whenever tuples are inserted, deleted, or modified.

---

5. Update Operations & Constraint Violations

5.1 The Three Operations

Operation	What it does	May violate...
INSERT	Adds a new tuple	Domain, Key, Entity Integrity, Referential Integrity
DELETE	Removes an existing tuple	Referential Integrity only (other tuples may reference the deleted PK)

Operation	What it does	May violate...
MODIFY (UPDATE)	Changes attribute value(s)	Domain, NOT NULL, Key, Referential Integrity (depends on which attribute)

5.2 Violation Actions

1. RESTRICT / REJECT — Cancel the operation; inform the user.
2. CASCADE — Propagate the change automatically to all referencing tuples.
3. SET NULL — Set the FK in referencing tuples to NULL.
4. User-defined error-correction routine.

> □ *One of these actions must be specified for each foreign key constraint at design time.*

5.3 Detailed Violation Rules

INSERT Violations

- Domain constraint — attribute value is not from the correct domain.
- Key constraint — PK value already exists in another tuple.
- Referential integrity — FK value does not match any PK in the referenced relation.
- Entity integrity — PK value is NULL.

DELETE Violations

- Can ONLY violate referential integrity.
- Occurs when the tuple's PK is referenced as an FK by another tuple elsewhere.
- Remedies: RESTRICT (reject delete), CASCADE (delete referencing tuples too), SET NULL (set FKs to null).

UPDATE (MODIFY) Violations

- Updating an ordinary attribute — can only violate domain constraint or NOT NULL.
- Updating the PK — treated like DELETE + INSERT; same options apply.
- Updating an FK — may violate referential integrity.

> ✓ *Exam Tip: UPDATE of a PK = DELETE old + INSERT new. Know this for exam MCQs.*

---

6. Quick Reference — All Constraints

Constraint	Applies To	Core Rule
Domain	Every attribute	Values must come from the defined domain (or be null if allowed)
Key / Superkey	Primary Key	PK must be unique across all tuples
Entity Integrity	Primary Key	PK can NEVER be null
Referential Integrity	Foreign Key	FK must match an existing PK or be null
Semantic	Application logic	Beyond the model; enforced by triggers / application code

> ✓*Exam Tip: Most exam questions on this chapter test: (1) informal-vs-formal terminology, (2) superkey vs. key vs. PK, (3) which constraint each operation can violate, and (4) the three remedies for DELETE violations.*

---

7. Worked Example — COMPANY Schema

Relations & Their Primary Keys

Relation	Primary Key	Notable Foreign Keys
EMPLOYEE	Ssn	Super_ssn → EMPLOYEE.Ssn; Dno → DEPARTMENT.Dnumber
DEPARTMENT	Dnumber	Mgr_ssn → EMPLOYEE.Ssn
DEPT_LOCATIONS	Dnumber + Dlocation	Dnumber → DEPARTMENT.Dnumber
PROJECT	Pnumber	Dnum → DEPARTMENT.Dnumber
WORKS_ON	Essn + Pno	Essn → EMPLOYEE.Ssn; Pno → PROJECT.Pnumber
DEPENDENT	Essn + Dependent_name	Essn → EMPLOYEE.Ssn

> □ *DEPT_LOCATIONS, WORKS_ON, and DEPENDENT all have composite primary keys.*

---

8. Key Formulas & Notation Cheat Sheet

Notation	Meaning
R(A1, A2, ..., An)	Schema of relation R with n attributes
r(R)	A specific state (population) of relation R
r(R) ⊆ dom(A1) × dom(A2) × ... × dom(An)	State is a subset of the Cartesian product of attribute domains
t[Ai] or t.Ai	Value of attribute Ai in tuple t
t[PK] ≠ null	Entity integrity — PK values are never null
t1[FK] = t2[PK] OR t1[FK] = null	Referential integrity rule for foreign keys

---